

RESEARCH ARTICLE

Design of High-Speed Multiway Merge Sorting Networks Using Fast Single-Stage N -Sorters and N -Filters

ROBERT B. KENT¹, (Life Member, IEEE), AND MARIOS S. PATTICHIS¹, (Senior Member, IEEE)

Department of Electrical and Computer Engineering, The University of New Mexico, Albuquerque, NM 87131, USA

Corresponding author: Robert B. Kent (rkent@unm.edu)

ABSTRACT In hardware such as FPGAs, Kenneth Batcher's Odd-Even Merge Sort and Bitonic Merge Sort are the state-of-the-art methodologies used to quickly sort a list of more than 16 input values. Both sorting networks feature merges of 2 sorted input lists into a single sorted output list. For both, a full sort of 64 and 512 input values requires 21 and 45 serial stages, respectively. Multiway merge sorting networks described here require significantly fewer serial stages. For example, 8-way merge networks fully sort 64 and 512 input values in 9 and 20 serial stages, less than half the number of the respective 2-way networks. When the multiway merge sorting networks utilize the single-stage N -sorters recently defined by the authors, they are considerably faster than Batcher's networks. In the AMD-Xilinx Ultrascale+ xcvu9p FPGA, the two 8-way merge networks have speedups of 1.85 and 1.74 versus the comparable 2-way networks. A fully pipelined 3-way merge network in this FPGA is capable of fully sorting 500 million lists of 729 unsorted 32-bit values in one second. In software, multiway merge methods are used to find the median of certain pixel rectangles in images, since the median can be determined in fewer stages than are required to fully sort the rectangle. However, the software still requires a series of many 2-sorter operations to find a median. These multiway merge median methods are dramatically sped up in hardware, where the authors' new single-stage N -sorters and N -filters operate in parallel in each stage of the merge process.

INDEX TERMS Sorting, sorting networks, multiway merge, FPGA, image filtering, median filter, N -sorters.

I. INTRODUCTION

The fundamental building block for a sorting system in hardware, such as an FPGA, has been the 2-sorter, which compares its 2 input values, and then uses the comparison result to select and transfer the inputs to the appropriate max and min output ports. When only 2 input values are to be sorted, a single 2-sorter is all that is needed.

When there are more than 2 input values to be sorted, networks containing 2-sorters operating in parallel have traditionally been used to sort the input list. Certain manually designed sorting networks using 2-sorters had previously been shown to produce the fastest sorting for up to 16 unsorted input values [1].

The associate editor coordinating the review of this manuscript and approving it for publication was Cesar Vargas-Rosales¹.

However, the authors have recently introduced single-stage N -sorters, with $16 \geq N \geq 3$, which are significantly faster than the previous state-of-the-art 2-sorter networks [2], [3]. For example, a single-stage 16-sorter design was shown to have a speedup of over 4.0 versus the equivalent custom-built 2-sorter network [3].

Single-stage devices have one set of inputs, one set of outputs, and whatever internal logic is required to present the output values in sorted order [2], [3]. Their speed superiority is due to the fact that they use only a single stage operation to sort an input list, versus the series of 2-sorter stages required by the comparable manually-designed sorting network.

If a single-stage device has N output ports, containing the full sorted list of the N inputs, we call the device an N -sorter. We refer to the device as an N -filter if there are less than N output ports. Examples of N -filters are N -max, N -min, and N -median filters.

For fast sorting of more than 16 unsorted input values, Kenneth Batcher's systematic sorting network algorithms, Odd-Even Merge Sort (O-EMS) and Bitonic Merge Sort (BMS) [4], have continued to be the state-of-the-art [5]–[10]. Both of Batcher's algorithms feature sequences of sorting stages which merge 2 sorted input lists into one sorted output list, using 2-sorters in each sequence stage.

In addition to sorting large unsorted input lists, Batcher's algorithms are also used to accelerate merge sort operations, by selecting and sorting several values from two large sorted input lists, which are then added to the single merge sort output list. This type of acceleration has been implemented in FPGA [8], CPU-SIMD [9], and GPU-SIMT [10] systems.

Here we introduce a multiway merge sorting network algorithm, which features merge sort sequences in which 3 or more sorted input lists are merged into a single sorted output list. Multiway merge sort sequences use our single-stage N -sorters to sort large input lists significantly faster than can be done using Batcher's 2-way merge algorithms. As will be discussed in detail in the background section, our multiway merge sorting network algorithm is related to algorithms previously proposed by other researchers, both for sorting full input lists in hardware and for determining the median of pixel windows in CPUs and in hardware.

We have combined the related algorithms and added to them, and present a consistent multiway merge sort algorithm for merging 3 or more sorted input lists. We call our methodology Unified Column Merge Sort, UCMS for short. The UCMS algorithm can also be used to build sorting networks in which 2 sorted lists are merged into one. However, these UCMS 2-way sorting networks are not novel. They are essentially the same as Batcher's O-EMS networks.

We also described in [3] how to design and implement fast and much larger single-stage N -max and N -min filters, and how they can be used in very fast N -max/ N -min sorting networks, which use few hardware resources. The N -max/ N -min single-stage and sorting network designs are fully covered in [3], and we do not expand on that coverage here.

Section II contains UCMS system definitions, and background is covered in Section III. UCMS networks are covered in detail in Section IV, including how they are constructed, and how their speed and resource usage compares to the state-of-the-art 2-way merge networks. UCMS filter networks, primarily median networks, are described in Section V. The main portion of this article concludes with Section VI, in which the speeds of both combinatorial and fully pipelined UCMS networks are discussed in detail. Appendix A shows how the total stages equation used in Section IV-E is derived.

In the associated supplemental material, the equivalence of Batcher's Odd-Even Merge Sort and UCMS 2-way merge sequences is shown. Also, a non-standard UCMS sorting network is described, and compared to the O-EMS network sorting the same number of values. An algorithm is then presented which shows how UCMS diagonal stage modules are constructed. This article also has a companion video, which

TABLE 1. UCMS Integer Parameter, Symbol, and Abbreviation Definitions.

Symbol	Definition
N_{total}	Total number of values in a full UCMS network
N_{cols}	Length of the unsorted input and sorted output lists Number of columns in a UCMS rectangle Number of sorted lists merged in a merge sequence in a UCMS "k-way" merge sequence, $k=N_{cols}$
N_{rows_0}	Number of rows in the Sequence 0 2-D array $2 \leq N_{rows_0} \leq N_{cols}$
q_{total}	Number of the merge sort sequences in the network Sequence q_{total} is the last sequence in the network
N_q	Number of values in a Sequence q rectangle
N_{rows_q}	Number of rows in a Sequence q rectangle
Num_rects_q	Number of rectangles in Sequence q
N -sorter	Hardware device that sorts N values at a time
SV	SystemVerilog, a Hardware Description Language
UCMS	Unified Column Merge Sort, this sorting system
O-EMS	Odd-Even Merge Sort; UCMS with $N_{cols}=2$
SS-M(k)	Multiway merge algorithm from Gao and Liu
VV	Early 4-way merge algorithm from Van Voorhis

Row		3	2	1	0	
7	--	32	31	30	29	q is the sequence number $N_{cols} = 4$; $N_{rows}_q = 8$ $N_q = N_{cols} * N_{rows}_q = 32$
6	--	28	27	26	25	
5	--	24	23	22	21	
4	--	20	19	18	17	Top row = ($N_{rows}_q - 1$) Bottom row = 0 Left column = ($N_{cols} - 1$) Right column = 0
3	--	16	15	14	13	
2	--	12	11	10	9	
1	--	8	7	6	5	
0	--	4	3	2	1	

FIGURE 1. Sorted Order of UCMS 4-way ($N_{cols}=4$) Merge Rectangle.

gives added detail to the example UCMS network construction which is presented in the early sections of Section IV.

II. DEFINITIONS

In UCMS merge sort sequences, the sorted input lists becomes the columns in a rectangular structure, hence the use of the variable name N_{cols} to refer to the number of sorted input lists. In each stage of the merge sequence, small single-stage N -sorters operate in parallel on the rectangle until the values in the rectangle are in sorted order after the final sequence stage. Table 1 contains definitions for UCMS rectangle parameters, as well other definitions related to this work, which are used throughout the discussions that follow.

Fig. 1 shows a 4-column, 8-row UCMS Sequence q rectangle example, and the final row-major sort order of distinct values 1 to 32 in that rectangle. The row max value is in the leftmost column, and the row min is in the rightmost column. Each column max value is in the top row, and the column min is in the bottom row. The overall array max value is in the at the top left of the array, and the overall min value is at the bottom right.

In this work, the term **propagation delay** refers to a sorting network's **worst-case propagation delay**, the time required for an input signal to propagate to an output along the slowest path in the network. We define the speedup ratio for a UCMS sorting network, relative to the comparable Odd-Even Merge

Sort network, as:

$$\text{Speedup} = \frac{\text{Prop_delay_O-EMS}}{\text{Prop_delay_UCMS_Network}}. \quad (1)$$

A sorting network's hardware resource increase, versus an O-EMS network, is defined as the resource increase ratio:

$$\text{Resource_ratio} = \frac{\text{Resources_UCMS_Network}}{\text{Resources_O-EMS}}. \quad (2)$$

III. BACKGROUND

The hardware sorting networks which are dominant today use 2-sorters operating in parallel in each stage of a sorting process. When sorting a small number N_{total} of unordered input values, from 3 up to 16, our single-stage devices have been shown to be much faster [3] than the previous state-of-the-art 2-sorter networks [1]. For fast sort of a large number of unsorted inputs, with $N_{total} > 16$, Kenneth Batcher's systematic sorting network algorithms [4] remain the state-of-the-art [5]–[10].

Several hardware multiway merge sorting processes have been proposed, which merge more than 2 sorted lists into a single sorted output list [11]–[13]. These multiway merge processes still use only 2-sorters in the merge process stages, and provide no speed improvement versus Batcher's 2-way merge networks. Two of these multiway merge algorithms [11], [12] do not appear to have any similarity to the UCMS algorithm proposed here. However, the final 2 stages in the Van Voorhis (VV) [13] algorithm's 4-way merge process appear to be equivalent to the final 2 stages in a UCMS $N_{cols}=4$ merge sort sequence, using rectangles with 4 columns.

One set of researchers did propose a systematic multiway merge sorting network algorithm, here called SS-M(k), for a prime number N_{cols} of sorted input lists to be merged [14]. The SS-M(k) algorithm calls for the use of single-stage N -sorters, with N from 2 up to N_{cols} , in the multiway merge process stages. Use of single-stage N -sorters allows the SS-M(k) merge sequences to use fewer stages than Batcher's comparable 2-way merge sequences. However, [14] did not provide a method to construct the needed single-stage N -sorters with $N \geq 3$.

Although the SS-M(k) algorithm was said to be valid when N_{cols} is a prime number, we have also seen that the algorithm works correctly when $N_{cols}=9$. This suggests that the basic SS-M(k) algorithm works when $N_{cols}=2$, and when N_{cols} is odd.

In its merge of several sorted input lists, the SS-M(k) algorithm constructs rectangular structures, with each input list becoming a column in the rectangle. We have adopted this methodology, and use the variable name N_{cols} to refer to the number of sorted input lists. There is no upper limit to the N_{rows_q} number of rows in an SS-M(k) rectangle, but $N_{rows_q} \geq 2$. As with UCMS, the rectangle cells have a defined order when fully sorted, and the merge stages operate on the rectangle values until they are in the correct rectangle sorted order, after the final merge process stage.

The primary problem with the SS-M(k) algorithm is that it, as is, does not work for any even $N_{cols} \geq 4$. A hint of how the SS-M(k) algorithm could be modified, for even $N_{cols} \geq 4$, was shown in the earlier work by Van Voorhis [13]. In the last stage of a 4-way merge of 16 values, VV sorts the internal values in each of the rectangle's 2 internal rows. In the UCMS algorithm, we generalize this methodology, and sort the internal values of internal rectangle rows in the last merge stage for any even $N_{cols} \geq 4$. As a result, UCMS is able to merge any number of sorted lists in a merge sequence.

A multiway merge methodology proposed for use in CPUs, but similar to the SS-M(k) algorithm, was proposed more recently by Kolte, et al [15], [16]. The Kolte group was primarily targeting finding the median of pixel windows, with $N_{rows_q} \leq N_{cols}$ and N_{cols} odd. However, they also proposed full sorting networks for both odd and even N_{cols} (still limited to $N_{rows_q} \leq N_{cols}$). It is not clear how their sorting networks with even N_{cols} values are implemented, but their methodology is not identical to that of VV/UCMS for a 4-way merge. The Kolte group reported using four additional 2-sorters to complete the full sort of a 4×4 rectangle [15], while the VV/UCMS method only requires two additional 2-sorters.

Kolte, et al, only use 2-sorters in all of their merge sequence stages, and they reported 20 additional 2-sorter operations to complete the sort of a 6×6 rectangle. It's not clear if these 20 additional operations required additional stages. In UCMS, the completion of a 6×6 rectangle sort is accomplished much more simply, by using 4 parallel single-stage 4-sorters in the last sequence stage.

After a UCMS rectangle is constructed using N_{cols} sorted lists, the first stage in a merge process consists of sorting the rectangle rows. This is true for the SS-M(k) and Kolte algorithms, as well as additional CPU algorithms that target finding pixel rectangle medians [6], [17]. After this row sort stage, both the columns and the row are still sorted [18]. When only looking for the rectangle median, this feature can be used to eliminate rectangle cells which cannot possibly provide the rectangle median, after which only the cells that can provide the median value are processed [6], [15]–[17].

The ability to ignore rectangle locations which cannot provide the median values is not the only feature of the algorithms which uses merge sequences to find medians. Median algorithms typically take advantage of the fact that, when N_{cols} is odd, determining the median takes fewer sequence stages than what are required to fully sort the input values. This is discussed in detail for UCMS merge sequences in Section V.

Even when the CPU algorithms [6], [15]–[17], take advantage of rectangle location elimination and stage reduction when determining median values, they suffer fundamental speed issues when transferred to hardware, as only 2-sorters are used throughout their merge processes. A UCMS median merge sequence may have as many stages in principle as a CPU algorithm, but the UCMS sequence is still much faster. UCMS uses fast single-stage N -sorters in each stage, but the

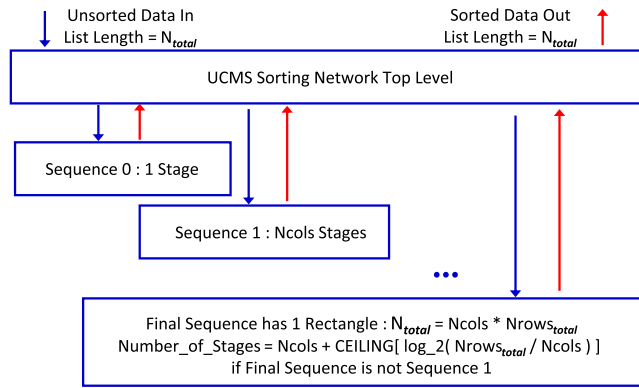


FIGURE 2. Standard UCMS Networks Top Level Diagram.

CPU requires a series of 2-sorters in each stage when sorting more than 2 input values.

Although several researchers mentioned here have described algorithms which are similar to UCMS, only the UCMS algorithm merges any number N_{cols} of sorted input lists, and any N_{rows_q} input list length. We have unified and added to the work of the earlier researchers, and have produced a systematic multiway merge methodology that can be applied to any large number of input values to be sorted.

IV. UNIFIED COLUMN MERGE SORT

To simplify UCMS design methodology, analysis, and discussion, we first define a specific class of networks, called UCMS standard sorting networks. UCMS standard sorting networks are designed in a straightforward manner, which allows them to be automatically generated using software. These standard networks are also easily analyzed, which enables development of equations for the total number of sorting network stages for standard UCMS networks. A top level diagram for UCMS standard sorting networks is shown in Fig. 2. Although non-standard UCMS sorting networks are discussed briefly in Section IV-H, the remainder of Section IV focuses on UCMS standard sorting networks.

A. STANDARD UCMS NETWORKS: OVERVIEW

A standard UCMS sorting network consists of 1 or more merge sort sequences, plus an initial stage which creates the sorted lists for the first merge sort sequence. In each merge sequence, N_{cols} sorted input lists are merged into a single sorted output list. In a standard UCMS merge Sequence q , each input list to be merged has the same length, labelled N_{rows_q} . Sorted input lists are arranged as the columns in a 2-D rectangle, which clearly has N_{cols} columns and N_{rows_q} rows. Several stages of parallel sorting operations are then used to put rectangle cell values into sorted order.

In a Sequence q rectangle, there are N_{rows_q} rows, the total number of the values is N_q , and so

$$N_q = N_{rows_q} \cdot N_{cols}. \quad (3)$$

In the final merge sequence, Sequence q_{total} , there is only one rectangle. The total number of values in the final rectangle, which is also the total number of values in the input and output lists, is called N_{total} , with

$$N_{total} = N_{rows_{q_{total}}} \cdot N_{cols}. \quad (4)$$

A UCMS standard network is defined by a set of four integer parameters, all of which are defined in Table 1, that satisfy Equation (5):

$$N_{total} = N_{rows_0} \cdot N_{cols}^{q_{total}}. \quad (5)$$

A set of parameters that satisfy Equation (5) is $(N_{total}, N_{rows_0}, N_{cols}, q_{total}) = (32, 2, 4, 2)$. The final sorted rectangle for this standard UCMS network matches the 4-column configuration of the rectangular array in Fig. 1.

The standard UCMS network with this particular set of parameters will be used as an example of how UCMS networks are constructed, and will be discussed in detail starting in Section IV-B. Although this particular 4-column network is discussed in detail, all of the numbered equations, starting with (3)-(5) above and continuing through (15) in Section IV-D, are valid for any UCMS sorting network.

The first part of the accompanying video can be used to follow along with the UCMS 4-column example covered in the next 3 subsections. The last part of the video shows how the sort groups and passthroughs are defined for the Sequence 2 diagonal stages shown in Section IV-D's Fig. 5.

B. STANDARD UCMS NETWORKS: SEQUENCE 0

The first set of parallel operations in a standard UCMS sorting network is found in the single stage of Sequence 0. In Sequence 0, the unsorted input list of length N_{total} is broken up into lists of length N_{rows_0} , with each list then being sorted. Sequence 0 is the only UCMS sequence that is not a merge sort sequence. The number of small Sequence 0 lists is simply:

$$Num_seq_0_lists = N_{total} / N_{rows_0}. \quad (6)$$

Each Sequence 0 list is sorted by an N_{rows_0} -sorter. When $N_{rows_0} > 2$, it is assumed that the N_{rows_0} -sorters are the single-stage devices introduced in [2], [3].

Fig. 3 shows the Sequence 0 operation for the example standard UCMS sorting network, with parameter set $(N_{total}, N_{rows_0}, N_{cols}, q_{total}) = (32, 2, 4, 2)$. The top rectangle in Fig. 3 shows how the 1-D input list of 32 values is placed into a 2-D rectangle with $N_{total}/N_{rows_0} = 32/2 = 16$ columns and $N_{rows_0} = 2$ rows. The Fig. 3 bottom rectangle shows the cell values after the Sequence 0 operation, in which each of the 16 columns is sorted by a 2-sorter, with maximum value placed at row $N_{rows_0} - 1$ and the minimum value placed at row 0. The red arrows in Fig. 3 indicate that all of the Sequence 0 sorting devices in this example are 2-sorters. All of the arrows used in Fig. 3 through 5 point from the location where the sort group minimum will be placed to the location where the max group value will be placed.

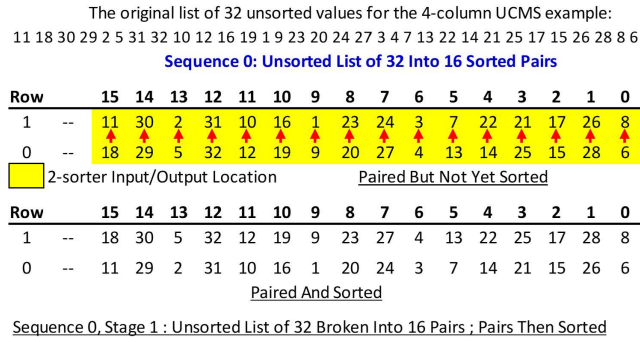


FIGURE 3. 4-way Merge UCMS Example: Sequence 0.

C. STANDARD UCMS NETWORKS: SEQUENCE 1

Sequence 1 is the first merge sort sequence in a standard UCMS sorting network. If $N_{total} \leq N_{cols}^2$, Sequence 1 is the only merge sort sequence.

The sorted lists which are the outputs of Sequence 0 are collected into groups of N_{cols} lists, with each of the sorted input lists becoming a column in a Sequence 1 rectangle. The number of rows in each Sequence 1 rectangle is then

$$N_{rows1} = N_{rows0}. \quad (7)$$

The number of Sequence 1 rectangles is

$$Num_rects1 = N_{total} / (N_{cols} \cdot N_{rows0}). \quad (8)$$

From Equation (3), the total values in each Sequence 1 rectangle is therefore

$$N_1 = N_{rows1} \cdot N_{cols} = N_{rows0} \cdot N_{cols}. \quad (9)$$

In the first stage of Sequence 1, and any merge sequence, each row in each rectangle is sorted using an N_{cols} -sorter. The maximum value in each row goes to column $N_{cols}-1$ and the minimum value goes to column 0.

As shown in Fig. 4 for the 4-way merge example Sequence 1, the 16 Fig. 3 sorted lists from Sequence 0 are broken up into $16/4=4$ groups, and therefore 4 rectangles, with 2 rows in each rectangle. As there are 4 values in each row, 4-sorters are used for the Stage 1 parallel row sort operations in Fig. 4. The blue arrows in the top set of Fig. 4 rectangles indicate that 4-sorters are the sorting devices that are used for the row sort stage.

All stages after the initial row sort stage in any merge sequence are diagonal stages, in which the values to be sorted in each sort group are selected along a diagonal. Starting at the lowest cell in a diagonal sort group, additional cells in the sort group are selected R rows up and C columns to the right from the previous cell. When a diagonal sort group is sorted, the minimum values goes to the lower left location in the diagonal, and the maximum value goes to the upper right location in the diagonal.

The diagonals in the first Sequence 1 diagonal stage, Stage 2, are selected along R/C 1/1 diagonals. In Fig. 4, the set of Stage 2 sort operations are shown for the 4 rectangles

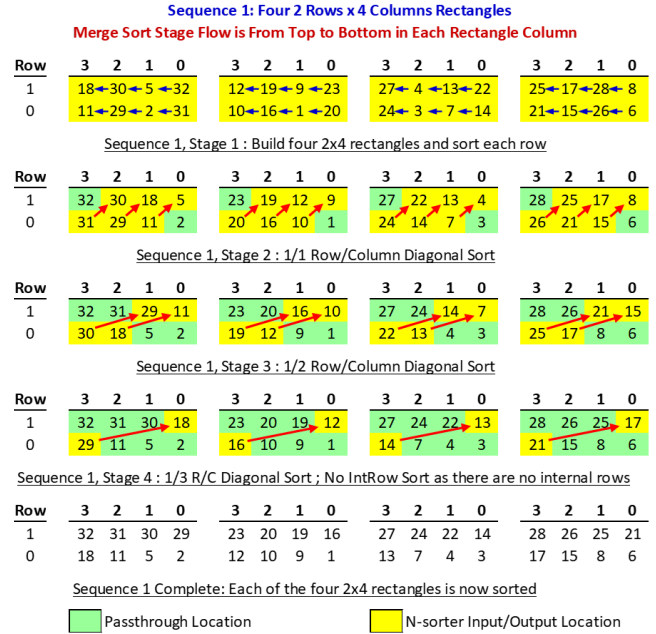


FIGURE 4. 4-way Merge UCMS Example: The 4 Stages of Sequence 1.

TABLE 2. UCMS Sequence 1 Stage Diagonal R/C Deltas and Row Sorts.

Column Delta	Ncols = 2	Ncols = 3	Ncols = 4	Ncols = 5	Ncols = 6	Stage
1	All	All	All	All	All	Row Sort
Ncols-4	1/1	1/1	1/1	1/1	1/1	R/C Diag
Ncols-3				1/2	1/2	R/C Diag
Ncols-2			1/2	1/3	1/4	R/C Diag
Ncols-1		1/2	1/3	1/4	1/5	R/C Diag
Ncols-1			IntRow		IntRow	Row Sort
Stages	2	3	4	5	6	

below the Stage 1 row sort operations. As there are only 2 rows in the Fig. 4 rectangles, 2-sorters are used for the 1/1 R/C Stage 2 operations, as shown by the red arrows for those 4 rectangles.

All diagonal stages have “passthrough” locations, rectangle locations which are not involved in an N -sorter operation. At these locations, input values are simple passed through to the output ports at that rectangle location. These locations are highlighted in green in the diagonal sort rectangles.

If there are additional stages after the R/C 1/1 stage, the row delta is always 1, and the column delta increments by 1 in each successive stage. The final stage in the merge sort sequence is a R/C $1/(N_{cols}-1)$ diagonal stage. As seen in Fig. 4, the two sequence stages after the 1/1 R/C stage are 1/2 and 1/3 R/C diagonal stages. As $N_{cols}-1=3$, the 1/3 R/C stage is the last stage in Sequence 1. All of the sorting devices in the last 2 stages are 2-sorters, indicated by the red arrows in those stage rectangles.

A Sequence 1 merge sort sequence always has N_{cols} stages. As can be seen in Fig. 4, this $N_{cols}=4$ Sequence 1 does have 4 stages.

Table 2 lists information for Sequence 1 stages, for N_{cols} values from 2 to 6. The last two lines in the table, with yellow highlight, both refer to the last sequence stage, when the column delta $C=N_{cols}-1$.

As mentioned previously, when N_{cols} is even and ≥ 4 , UCMS sorts the internal values of internal rectangle rows in the last sequence stage. This special row sort is called IntRow sort, and the specifics of its use is shown in the last line in Table 2. Although IntRow sort is indicated when $N_{cols}=4$, there is no IntRow sort shown in Fig. 4. The reason for this is that there are only 2 rows in the Fig. 4 Sequence 1 rectangles. There are no internal rows.

D. STANDARD UCMS: SEQUENCES 2 AND HIGHER

If $N_{total} > N_{cols}^2$, there will be at least one more merge sort sequence after Sequence 1. For a UCMS standard sorting network, the final sequence number q_{total} has already been defined, since it is a required parameter for a UCMS standard network definition.

Each merge sequence after Sequence 1 will have taller rectangles, but fewer of them. More specifically, for Sequence q with $q > 1$,

$$N_{rows_q} = N_{rows_{q-1}} \cdot N_{cols}, \quad (10)$$

and

$$Num_rects_q = \frac{Num_rects_{q-1}}{N_{cols}}. \quad (11)$$

Using Equations (7) and (10), it can readily be determined that

$$N_{rows_q} = N_{cols}^{q-1} \cdot N_{rows_0}. \quad (12)$$

Likewise, using Equations (8) and (11), it can also be derived that

$$Num_rects_q = \frac{N_{total}}{N_{cols}^q \cdot N_{rows_0}}. \quad (13)$$

For the 4-way UCMS Sequence 2 example shown Fig. 5, these equations resolve to

$$\begin{aligned} N_{rows_2} &= N_{cols} \cdot N_{rows_1} = 4 \cdot 2 = 8. \\ Num_rects_2 &= 32 / (4^2 \cdot 2) = 32 / (16 \cdot 2) = 1. \end{aligned}$$

In Fig. 5, the first Sequence 2 Stage 1 is the row sort stage, followed by the first diagonal stage, Stage 2. In Stage 2, the diagonal selections have an R/C row/column delta value of 2/1. For clarity, one of the 4-sorter groups in the Stage 2 is highlighted in pink in Fig. 5.

The locations at row 7, column 3 and row 0, column 0 are passthrough locations for all diagonal stages. These are rectangle max and min locations, respectively. Both the rectangle max and min values were determined during Stage 1, the row sort stage.

Fig. 5's Stage 3, like Stage 2 in Fig. 4, is a R/C 1/1 stage. Likewise, the 1/2 and 1/3 R/C diagonals in Fig. 5's Stage 4 and 5 match the Stage 3 and 4 diagonals in Fig. 4.

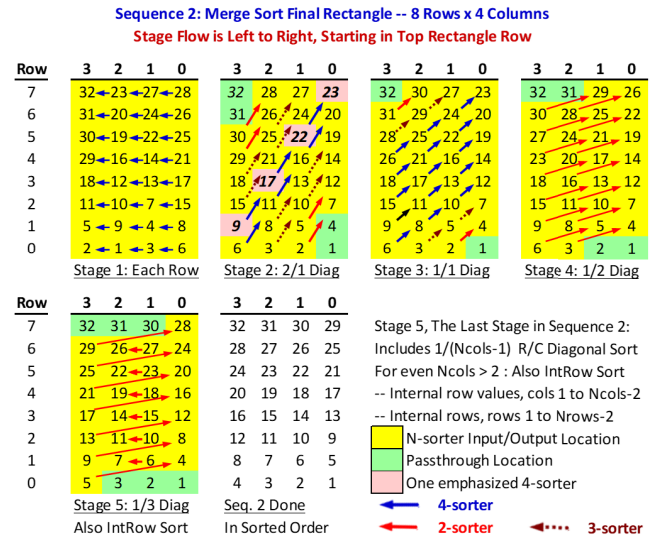


FIGURE 5. 4-way Merge UCMS Example: The 5 Stages of Sequence 2.

Note that final Fig. 5 Sequence 2 stage does have IntRow sort operations. The 2 internal values in each of the 6 internal rows are sorted, with the maximum value going to column $N_{cols}-2$, and the minimum value going to column 1.

Any Sequence q with $q \geq 2$ has the same stages as Sequence 1, but 1 or more extra stages between the Stage 1 row sort stage and the R/C 1/1 stage. All of the extra stages are also diagonal stages, with a column delta of 1, and a row delta that is > 1 and an integer power of 2. In Fig. 5, there is just 1 extra stage, the R/C 1/2 diagonal Stage 2.

In a Sequence q after Sequence 1, the number of extra stages required to sort N_q values is given by:

$$Num_Extra_Stages_q = \text{CEILING} \left(\log_2 \left(\frac{N_{rows_q}}{N_{cols}} \right) \right). \quad (14)$$

The first extra stage in any sequence after Sequence 1, which will be Stage 2 in that sequence, will have the maximum row delta of any stage in the sequence, and is given by:

$$Stage_2_Row_Delta_q = 2^{Num_Extra_Stages_q}. \quad (15)$$

If there are other extra stages after Stage 2, each extra stage will have a row delta one half that of the preceding extra stage. The last extra stage will have a row delta of 2.

For the example Sequence 2 shown in Fig. 5, Equations (14) and (15) become

$$\begin{aligned} Num_Extra_Stages_2 &= \left\lceil \log_2 \left(\frac{N_{rows_2}}{N_{cols}} \right) \right\rceil = \left\lceil \log_2 \left(\frac{8}{4} \right) \right\rceil = 1. \\ Max_Row_Delta_2 &= Stage_2_Row_Delta_2 = 2^1 = 2. \end{aligned}$$

We have developed an algorithm which can be used for the creation of any diagonal stage module in an HDL, a Hardware Description Language. The algorithm is presented in the associated supplemental material. The last section of the companion video shows how the sort groups and passthroughs

TABLE 3. Equations for Total Serial Stages, By N_{cols} , By $\log_{N_{cols}}(N_{total})$.

N_{cols}	$N_{cols} = 3, 5, 6, 7, 9$	$N_{cols} = 2, 4, 8$
2		$\frac{1}{2} \cdot [(1 \cdot p^2) + (1 \cdot p)]$
3, 4	$\frac{1}{2} \cdot [(2 \cdot p^2)]$	$\frac{1}{2} \cdot [(2 \cdot p^2) + (2 \cdot p) - 2]$
5	$\frac{1}{2} \cdot [(3 \cdot p^2) + (1 \cdot p) - 2]$	
6	$\frac{1}{2} \cdot [(3 \cdot p^2) + (3 \cdot p) - 4]$	
7, 8	$\frac{1}{2} \cdot [(3 \cdot p^2) + (5 \cdot p) - 6]$	$\frac{1}{2} \cdot [(3 \cdot p^2) + (7 \cdot p) - 8]$
9	$\frac{1}{2} \cdot [(4 \cdot p^2) + (6 \cdot p) - 8]$	

In the last column, when N_{cols} is a power of 2, the equations are exact.
In the first equations column, the equations produce upper bound values.

for the diagonal stage modules in Fig. 5 are constructed, as specified in the algorithm.

E. ANALYSIS: THE NUMBER OF UCMS STAGES

The SS-M(k) researchers [14] presented an equation for the number of serial stages in their sorting networks, when N_{total} is an integer power of N_{cols} . This equation, and its derivation, is discussed in detail in Appendix A.

In Table 3, that equation is broken out into specific smaller equations, based on the value of N_{cols} . The equations in the rightmost column, when N_{cols} is a power of 2, produce the exact number serial stages for the associated networks. The equations in the first equations column, where N_{cols} is not a power of 2, are upper bound equations.

The equations in this table are presented in a manner which allows the various equations to be easily compared. The equation in Table 3's first row, for $N_{cols}=2$, can be rearranged to show that it is identical to the more recognizable

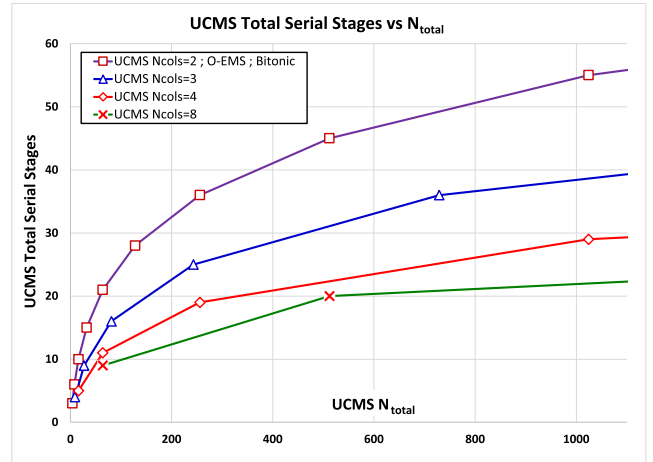
$$(1/2)p(p+1)$$

form that Batcher presented for Odd-Even Merge Sort and Bitonic Merge Sort [4].

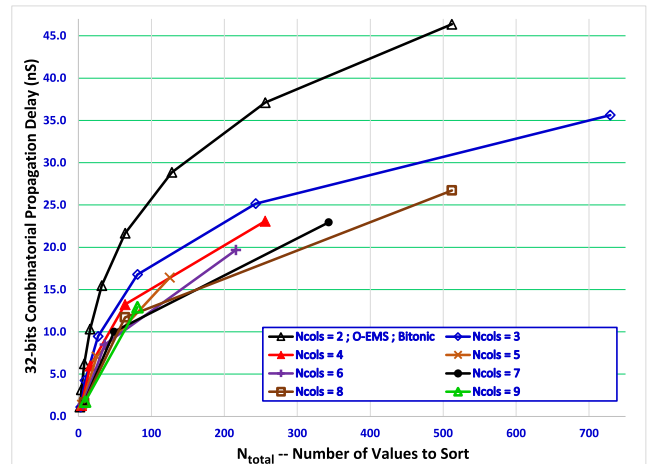
Fig. 6 uses the Table 3 equations to plot the UCMS total number of serial stages vs N_{total} , for $N_{cols}=2, 3, 4, 8$. Table 4 lists several values for $N_{cols}=2, 4, 8$ serial stages. The number of $N_{cols}=2$ stages are also the number of stages for Batcher's O-EMS and Bitonic Merge networks. Note that the number of $N_{cols}=2$ serial stages for $N_{total}=64; 512$ networks, 21 and 45 respectively, are over twice the 9 and 20 number of stages for the comparable $N_{cols}=8$ networks.

If all of the sorting devices in the Fig. 6 sorting networks had the same propagation delay, Fig. 6 would also display the behavior of total propagation delay vs N_{total} . However, the various sorting devices used in these networks do not have the same propagation delay, as was shown for the AMD-Xilinx xcvu9p FPGA analyzed in our earlier work [2], [3].

An N_{cols} sorting network uses N -sorters from 2-sorters up to N_{cols} -sorters, and the [2], [3] data showed that N -sorter propagation delay tends to increase as N increases. In the following section, we will use our N -sorters from [3] to plot propagation delay vs N_{total} , using synthesis data for UCMS networks implemented in the same target FPGA used in [3].

**FIGURE 6.** UCMS Total Serial Stages vs N_{total} ; $N_{total}=N_{cols}^P$.**TABLE 4.** Stages in Series For $N_{cols}=2, 4, 8$ UCMS Networks.

$N_{total} ==>$	16	64	256	512	1024
Sorting Network					
$N_{cols}=2$	10	21	36	45	55
$N_{cols}=4$	4	11	19		29
$N_{cols}=8$		9		20	

**FIGURE 7.** UCMS Network Speed vs N_{total} ; $N_{total}=N_{cols}^P$.

F. STANDARD UCMS SPEED AND RESOURCE USAGE

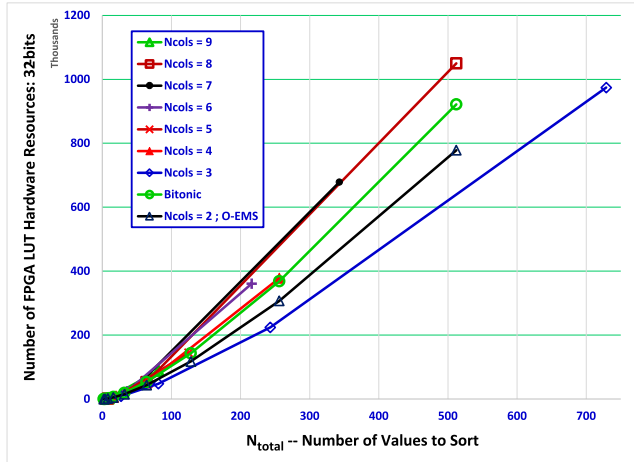
Speed and resource usage data has been gathered for standard UCMS sorting networks, using synthesis results produced by AMD-Xilinx Vivado 2018.2 software. The single-stage N -sorters used in the UCMS network designs are generally the fastest N -sorters that were constructed in [3]. These N -sorters were defined for the AMD-Xilinx Ultrascale+ xcvu9p product, the FPGA used in Amazon's AWS EC2 F1 instance [19], [20].

Fig. 7 displays speed data for UCMS sorting networks with N_{cols} values ranging from 2 up to 9. The raw data plotted in Fig. 7 is listed in the first table in the supplemental

TABLE 5. Prop Delay Speedup For $N_{cols}=4, 8$ vs $N_{cols}=2$ Networks.

$N_{total} ==>$	16	64	256	512
Sorting Network				
$N_{cols}=4$	1.728	1.636	1.606	
$N_{cols}=8$		1.851		1.736

Prop Delay Speedup = Prop_Delay_Ncols_2 / Prop_Delay_Comparison .

**FIGURE 8.** UCMS LUT Resource Usage vs N_{total} ; $N_{total}=N_{cols}^p$.

material. The values in the various UCMS sorting networks are 32-bit unsigned integers. The N_{total} numbers plotted are those that are the most efficient for a given N_{cols} value, with $N_{total}=N_{cols}^p$, and power variable p an integer ≥ 2 .

The $N_{cols}=2$ speed curve in Fig. 7 is assumed to represent Batcher's Bitonic Merge Sort as well as O-EMS. The $N_{cols}=2$ networks are clearly the slowest, and $N_{cols}=3$ curve also indicates slower operation than the higher N_{cols} networks. The $N_{cols}=4$ curve may be slightly slower than the $N_{cols} \geq 5$ networks, and the speed of the $N_{cols}=5$ to 9 networks appear to be roughly the same.

Table 5 lists the propagation delay speedup values for several $N_{cols}=4, 8$ UCMS networks versus $N_{cols}=2$ /O-EMS/Bitonic networks. The propagation delay speedups are impressive, from a minimum of 1.606 for $N_{cols}=4$ / $N_{total}=256$ networks, to a maximum of 1.851 for $N_{cols}=8$ / $N_{total}=64$ UCMS networks.

Fig. 8 plots the FPGA LUT resource usage for the standard UCMS networks whose speed data is plotted in Fig. 7. Fig. 8's raw data is listed in the second table in the supplemental material. The LUT resource usage data was taken from the same synthesis results used for the propagation delay data. In Fig. 8, there is a separate curve for Bitonic Merge networks, since they use more resources than O-EMS networks. Bitonic Merge networks were not run through synthesis, but their resource usage is easily calculated.

The UCMS $N_{cols}=3$ sorting networks consistently use fewer resources than the $N_{cols}=2$ /O-EMS networks. The

TABLE 6. LUT Ratio For Bitonic, $N_{cols}=4, 8$ vs $N_{cols}=2$ Networks.

$N_{total} ==>$	16	64	256	512
Sorting Network				
Bitonic	1.270	1.238	1.200	1.184
$N_{cols}=4$	1.121	1.176	1.231	
$N_{cols}=8$		1.214		1.349

LUT Increase Ratio = LUT_Usage_Comparison / LUT_Usage_Ncols_2 .

resource usage for $N_{cols}=4$ UCMS networks appear to be comparable to Bitonic Merge networks, at least for N_{total} up to 256.

Table 6 lists LUT resource increase ratios for Bitonic and $N_{cols}=4, 8$ networks versus $N_{cols}=2$ /O-EMS networks. Of note is the fact that the LUT resource usage for both $N_{cols}=4, 8$ networks at $N_{total}=64$ is less than Bitonic usage.

G. DEFINING A STANDARD UCMS SORTING NETWORK

Creating a standard UCMS sorting network begins with finding a set of $(N_{total}, N_{rows0}, N_{cols}, q_{total})$ parameters that satisfy Equation (5). As shown below, a straightforward method can be used to determine whether a parameter set is valid or not.

To find out if an $N_{cols}=3$ UCMS standard sorting network can be constructed for $N_{total}=32$, first determine q_{total} for this (N_{total}, N_{cols}) pair:

$$q_{total} = \text{CEILING}(\log_{N_{cols}}(N_{total})) - 1. \quad (16)$$

Specifically, for the $(N_{total}, N_{cols}) = (32, 3)$ pair,

$$\begin{aligned} q_{total} &= \text{CEILING}(\log_3(32)) - 1 \\ &= \text{CEILING}(3.15) - 1 = 4 - 1 = 3. \end{aligned}$$

To determine N_{rows0} :

$$N_{rows0} = N_{total} / (N_{cols}^{q_{total}}). \quad (17)$$

For the $(N_{total}, N_{cols}) = (32, 3)$ example,

$$N_{rows0} = 32/3^3 = 32/27 = 1.19.$$

N_{rows0} must be an integer, with $2 \leq N_{rows0} \leq N_{cols}$, so a standard UCMS sorting network cannot be constructed for the $(N_{total}, N_{cols}) = (32, 3)$ pair.

However, going through this exercise for $(N_{total}, N_{cols}) = (32, 4)$, we find that

$$\begin{aligned} q_{total} &= \text{CEILING}(\log_4(32)) - 1 \\ &= \text{CEILING}(2.5) - 1 = 3 - 1 = 2. \end{aligned}$$

$$N_{rows0} = 32/4^2 = 32/16 = 2.$$

In this case, N_{rows0} is an integer with $2 \leq N_{rows0} \leq N_{cols}$, so the parameter set $(N_{total}, N_{rows0}, N_{cols}, q_{total}) = (32, 2, 4, 2)$ does define a standard UCMS sorting network. This is, of course, the standard UCMS network that has been used as the UCMS example sorting network in earlier discussions.

It is also straightforward to determine that standard UCMS networks can be defined for $N_{total}=32$ when either $N_{cols}=2$ or $N_{cols}=8$.

H. NON-STANDARD UCMS SORTING NETWORKS

Thus far, we have focused on standard UCMS sorting networks, UCMS networks that satisfy Equation (5). Due to their systematic nature, standard UCMS sorting networks are able to be generated in a straightforward manner using software.

In addition, as described in Section IV-E a set of equations has been defined for the number of serial stages in a standard UCMS network, whenever N_{total} is an integer power of N_{cols} . An analysis based on these equations shows that UCMS sorting networks, with $N_{cols} \geq 3$, use many fewer serial stages, versus the existing state-of-the-art 2-way merge networks, which ultimately leads to a significant speed advantage for UCMS multiway merge networks.

However, non-standard UCMS networks, those which do not satisfy Equation (5), can still be quite useful. An example of such a non-standard UCMS sorting network, with $N_{total}=8$ and $N_{cols}=3$, is presented in the supplemental material, and is compared to the standard UCMS/O-EMS $N_{total}=8$ and $N_{cols}=2$ sorting network there. The $N_{cols}=3$ network requires 4 stages to sort the 8 values, while the O-EMS/ $N_{cols}=2$ network needs 6 stages. The $N_{cols}=3$ network uses 3-sorters as well as 2-sorters, but 3-sorters have been shown to be nearly as fast as 2-sorters [2], [3]. Therefore, it's reasonable to estimate that the non-standard $N_{cols}=3$ network has a speedup of $1.5=6/4$ versus the comparable standard O-EMS/ $N_{cols}=2$.

In another type of non-standard UCMS network, input sorted lists whose length is $>N_{cols}$ may be provided by single-stage N -sorters, rather than using the multiple stages of a standard merge sequence. This type of non-standard network is utilized and discussed in the next N -filters section.

V. UCMS N-FILTERS

A UCMS N -filter network produces a sorted list from N unsorted input values, but provides only a subset of the N values as outputs. Two obvious N -filter examples are N -max and N -min network filters. However, these simple filter networks were covered in [3], and are not discussed further in this work. Another obvious network N -filter is an N -median filter when N is odd. N -median filters will be discussed in detail here.

N -median filters are very popular for reducing noise in images. As mentioned in Section III, a number of CPU software researchers have targeted finding the median of square or rectangular pixel windows, and have used concepts similar to those in UCMS to minimize the work required to find the window median. In particular, when N_{total} and N_{cols} are both odd, the median can be found in fewer UCMS stages than what are required to produce a full sort of the N_{total} values. The stage reduction is 1 when $N_{cols}=3$, 2 when $N_{cols}=5$, and generally $(N_{cols}-1)/2$.

The original list of 25 unsorted values for the 5-column UCMS example is
18 10 22 14 6 12 25 8 19 13 4 17 16 24 9 23 2 21 7 1 3 11 15 5 20

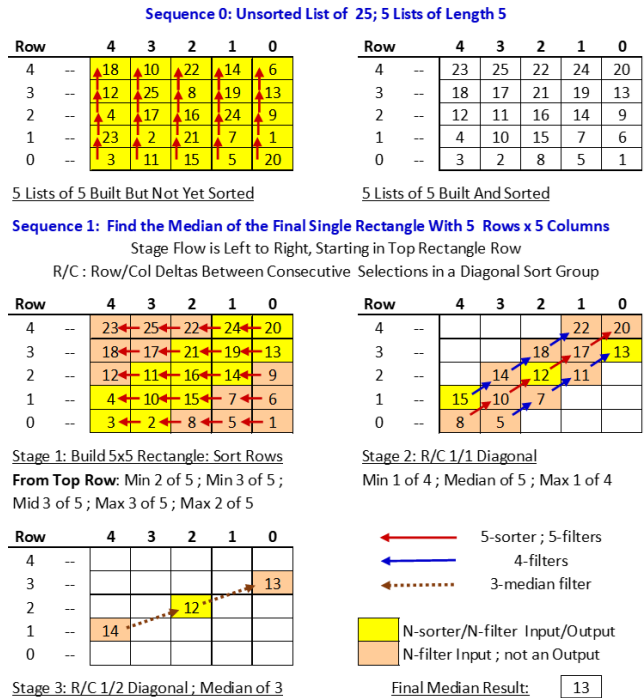


FIGURE 9. UCMS Median of 25 Values Using 5-way Merge.

Fig. 9 shows a 4-stage 25-median sorting network with $N_{cols}=5$. It has been noted by CPU researchers that, while 4 network stages are required to determine the 25-median using a 5×5 window [15], 6 network stages are required to produce a full 25-sort using this 5×5 window [16]. However, in the CPU implementations used in these publications, each of these stages require several serial substages, as only 2-sorters are used in the sorting process. In the comparable Fig. 9 UCMS 25-median sorting network, there are truly only 4 serial stages in the sorting process, as each stage consists of our single-stage N -sorters/ N -filters operating in parallel.

The Fig. 9 25-median design is similar to the standard UCMS network with $N=25$ and $N_{cols}=5$. However, the UCMS standard network outputs all 25 input values, uses only single-stage N -sorters instead of N -filters, and requires 2 more stages in addition to the 4 stages shown in Fig. 9.

One very recent image researcher has described several N -median algorithms using a CPU's 2-sorter operations [6]. For example, [6] defines two 27-median algorithms, one which takes 111 swaps and 222 load/stores, and another which takes 148 swaps but only 105 load/stores. The focus of [6] is on 16-bit data, as images increasingly use 16-bit pixels.

Here, we have implemented several different UCMS 27-median networks, using 16-bit values, and discuss and compare their performance in our target FPGA. We have 4 different designs, with $N_{cols}=2, 3, 5, 7$, shown in Figs. 10, 11, 12, and 13. Synthesis speed and resource usage data for these designs are shown in Table 7.

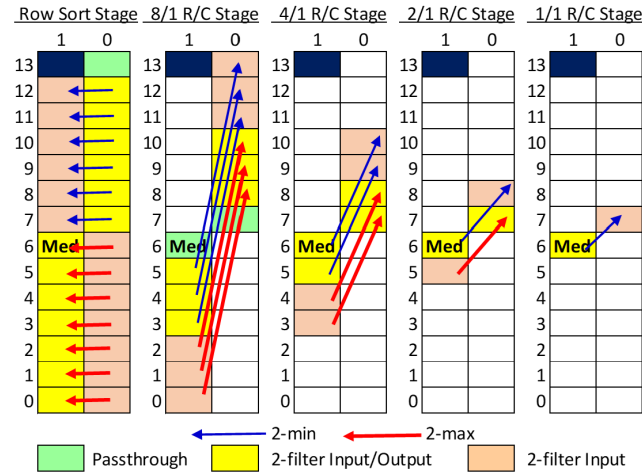


FIGURE 10. UCMS Median of 27 Values Using 2-way Merge.

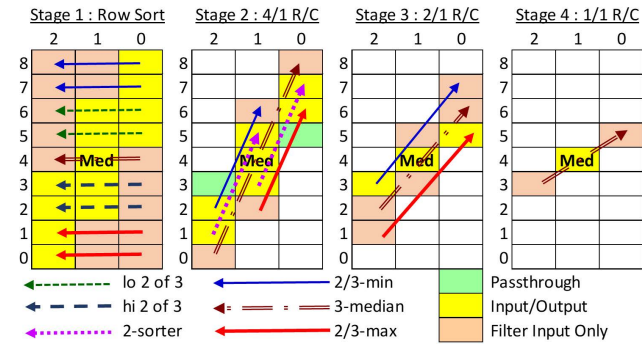


FIGURE 11. UCMS Median of 27 Values Using 3-way Merge.

These 4 27-median networks are clearly non-standard, and 3 of the 4 networks merge lists with differing lengths. The 2-way merge network in Fig. 10 merge lists of length 13 and 14, both of which are produced by single-stage 13- and 14-sorters. Using the single-stage 13-/14-sorters significantly improves the speed of the Fig. 10 design. As shown in Table 7, the 2-way merge 16-bit 27-median design has a propagation delay of 6.687 nS. As we reported in Table 2 of [3], an 8-bit 12-network using only 2-sorters was slower than this, at 6.872 nS. Table 7 does indicate that there is a resource usage penalty when using the 13-/14-sorters in the 2-way merge 27-median design.

The 3-way and 5-way designs shown in Figs. 11 and 12 also benefitted from merging taller input lists than what can be produced in a normal Sequence 0 for $N_{cols}=3$ or 5 standard UCMS networks. The 3-way, 5-way, and 7-way 27-median designs, all of which have 5 serial stages, are significantly faster than the 6-stage 2-way design, and use significantly fewer resources as well.

It is difficult to directly compare CPU and FPGA speeds, but the low propagation delays listed in Table 7 for the 27-median FPGA designs appear to be an improvement versus the swaps and loads/stores reported for the CPU designs in [6]. UCMS sorting network speeds will be discussed in more detail in Section VI.

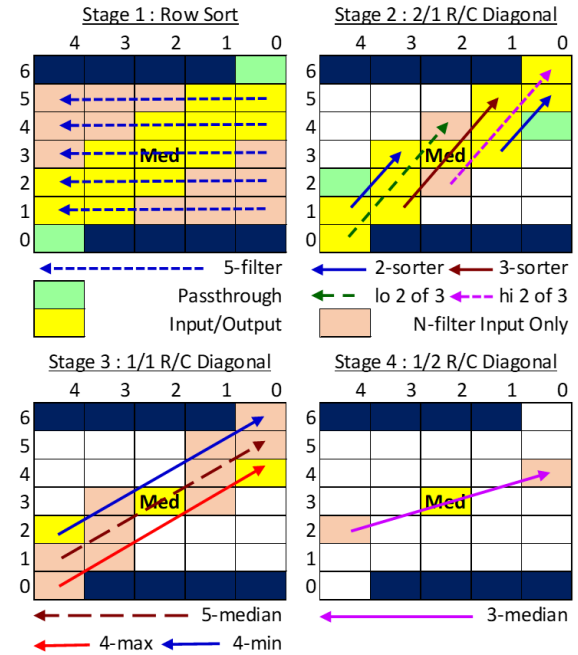


FIGURE 12. UCMS Median of 27 Values Using 5-way Merge.

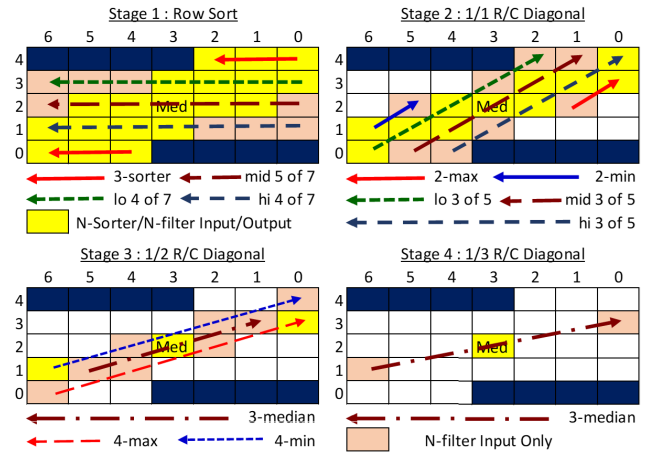


FIGURE 13. UCMS Median of 27 Values Using 7-way Merge.

Table 7 also lists speed and resource usage results for full 27-sorter designs. Interestingly, the 2-way merge design still requires only 6 serial stages, and is now the fastest of the 4 designs. Each of the 3-, 5-, and 7-way designs require more stages for a full 27-sorter design, and now the 3-way design is the only one of the three that is competitive with the 2-way design's speed.

VI. THE SPEED OF UCMS SORTING NETWORKS

When measuring the speed of UCMS sorting networks, this article focusses on the combinatorial delay of the slowest signals propagating from input to output ports. For optimum usage of UCMS networks, we emphasize networks which process more than 16 inputs, since our single-stage N-sorters and N-filters can quickly process up to 16 values.

In the AWS EC2 F1 instances we deal with here, clocks can operate as fast as 500 MHz [20]. The combinatorial

TABLE 7. 27-Median/27-Sort UCMS 16-bit Prop Delay and Resource Usage.

UCMS Design	N_{cols}	Stages	Standard Stages	LUT Resources	Prop Delay (nS)
Median_27	2	6	15	11467	6.686
Median_27	3	5	8	5312	5.728
Median_27	5	5	10	4272	5.853
Median_27	7	5	5	4544	6.200
Sort_27	2	6	15	13003	6.810
Sort_27	3	6	9	7024	6.837
Sort_27	5	7	12	7504	7.935
Sort_27	7	8	8	7936	9.278

Standard Stages: Total stages using a standard Seq. 0/Seq. 1/etc. flow.

propagation delays of the UCMS networks we describe here are all above 2 nS, so none of them will complete operation within a single 500 MHz clock period.

However, it may be unrealistic to provide a new set of N_{total} inputs in every 2 nS clock period, particularly if the N_{total} inputs are streamed into the FPGA from the host CPU using the AWS PCIe data transfer system. When data is streamed in and out of the FPGA using PCIe, the speed of operation is typically limited by the speed of the PCIe data transfer, not the inherent speed of the combinatorial UCMS sorting network.

The base AWS EC2 F1 system can transfer data at up to 10 Gbps, and the largest F1 instance transfers data at 25 Gbps [19]. The AWS PCIe system operates at 250 MHz [20], so the data transfer rates become $40=10,000/250$ and $100=25,000/250$ bits per 4 nS period, respectively.

To shift in 27 16-bit values for the 27-median and 27-sorter designs previously discussed, it will take $11=\text{CEILING}(27 \cdot 16/40)$ 4 nS periods for the 10 Gbps system, and $5=\text{CEILING}(27 \cdot 16/100)$ periods for the 25 Gbps system. All of the 27-sorters and 27-median 16-bit propagation delays in Table 7 are less than 10 nS, much less than the 20 nS required to transfer 27 16-bit values in the fastest 25 Gbps system.

After a set of 27 input values has been captured at time 0 at the inputs to a combinatorial 27-sorter/27-filter, the set of inputs can be easily sorted in the 20 nS required to shift in a new set of 27 inputs, using the fastest PCIe system. At time 20 nS, a second set of 27 input values is captured at the sorting network inputs, and the sorted values from the time 0 27 inputs are captured at the network outputs. Between time 20 nS to 40 nS, a third set of 27 inputs is shifted in, the sorted outputs from the first set of 27 inputs is shifted out to the CPU, and the second set of 27 inputs is sorted. And so on.

In short, the combinatorial UCMS sorting network is not the limiting factor for speed of operation. The UCMS sorting networks can sort lists of data as fast as the PCIe system can transfer those lists in and out of the FPGA.

If it is possible to provide a new set N_{total} inputs every 2 nS, the UCMS sorting network can operate in fully pipelined mode in order to sort the N_{total} values. In an FPGA, any combinatorial sorting network can easily be converted to

fully pipelined operation, in which a new set of inputs can be applied to the sorting network in each clock cycle. The throughput of the UCMS sorting network then becomes N_{total} inputs every 2 nS clock period. The latency of the fully pipelined sorting network then equals the total number of network serial stages multiplied by the 2 nS clock period.

The implementation of a fully pipelined UCMS sorting network is possible as long as the sorting process for each network stage takes less than the 2 nS clock period of the 500 MHz system. All of the single-stage N -sorters and N -filters discussed here have a propagation delay of less than 2 nS, so all of the UCMS sorting networks which can be implemented in the target FPGA can be fully pipelined at 500 MHz.

While gathering the synthesis speed and resource data for the analysis in Section IV-F, we noted that a $N_{cols}=3$; $N_{total}=729$ 32-bit standard UCMS network could be constructed using 974256 LUTs, fewer than the 1182240 LUTs available in the xcvu9p FPGA. Since this UCMS network can be implemented in the target FPGA, the fully pipelined version of this network can then sort 500 million lists of 729 32-bit values in one second.

VII. CONCLUSION

Unified Column Merge Sort, which uses multiway merge sequences to fully sort an unsorted input list of N_{total} values, has been described in detail in this work. Its use of N_{cols} -way merge, with $N_{cols} \geq 3$, enables UCMS networks to use many fewer serial stages than the existing state-of-the-art 2-way merge sorting networks. UCMS networks combining this smaller set of serial stages with the recently introduced single-stage N -sorters have been shown to be significantly faster than the state-of-the-art 2-way merge networks, when implemented in the xcvu9p FPGA found in the Amazon AWS EC2 F1 instance.

For example, a UCMS 8-way merge network needs 20 serial stages to sort 512 input values, while a comparable state-of-the-art Odd-Even Merge Sort requires 45 serial stages. When the two network types are synthesized for the target FPGA, the UCMS 8-way merge network has a speedup of 1.74 versus the O-EMS network.

In CPU algorithms that correspond to a portion of the UCMS algorithm, it has been shown that finding the median of a pixel rectangle with odd numbers of both rows and columns can be done more quickly than fully sorting the rectangle. However, the CPU uses a series of many 2-sorter operations to find the median, while UCMS can determine the median much more quickly using single-stage N -sorters and N -filters operating in parallel.

When FPGA input and output data is streamed via the fastest AWS PCIe interface, combinatorial UCMS filters and sorting networks will sort data as fast the 250 MHz interface can stream the data to and from the FPGA. If, however, data is available at the FPGA without requiring PCIe streaming, UCMS networks can operate at high speed using fully pipelined operation. Because of the speed of the

single-stage N -sorters and N -filters, UCMS networks can be fully pipelined at 500 MHz, the maximum speed of AWS instance. It has been shown that a fully pipelined UCMS 3-way merge network in the AWS instance can fully sort 500 million input lists in one second, with each list containing 729 unsorted 32-bit values.

APPENDIX A

STANDARD UCMS TOTAL STAGES

The SS-M(k) researchers presented an equation for the total number of stages in what we call a UCMS standard sorting network, for any N_{cols} and when N_{total} is an integer power p of N_{cols} : $N_{total} = N_{cols}^p$ [14]. Using our UCMS terminology, this equation is

$$Total_Stages = 1 + (p-1)N_{cols} + \frac{1}{2}(p-2)(p-1) \text{CEILING}(\log_2 N_{cols}). \quad (18)$$

In Section IV-E, this equation is broken out in Table 3 for each $9 \geq N_{cols} \geq 2$, and the equations for four N_{cols} values are used to produce the data plotted in Fig. 6.

It is reasonable to focus on N_q values which are an integer power of N_{cols} , because this is when UCMS sorting networks are most efficient. For example, the 4-way rectangles in Fig. 4 used all 4 of the Sequence 1 stages to sort 8 values. However, if N_{rows_0} had been 4, the Sequence 1 rectangles would have had $N_1 = N_{rows_0} \cdot N_{cols} = N_{cols}^2 = 16$ values, while still needing only the same 4 Sequence 1 merge sort stages that the $N_1 = 8$ rectangles required.

To derive (18), the first step is to evaluate N_{rows_p} , using Equation (3):

$$N_p = N_{cols}^p = N_{rows_p} \cdot N_{cols}.$$

So

$$N_{rows_p} = N_{cols}^p / N_{cols} = N_{cols}^{p-1}.$$

The number of extra stages then becomes

$$\begin{aligned} Number_Extra_Stages_p &= \text{CEILING} \left(\log_2 \left(\frac{N_{cols}^{p-1}}{N_{cols}} \right) \right) \\ &= \text{CEILING} \left(\log_2 (N_{cols}^{p-2}) \right) \\ &= \text{CEILING} \left[(p-2) \log_2 (N_{cols}) \right] \end{aligned} \quad (19)$$

This equation is made into an upper bound equation by moving the $(p-2)$ term outside of the CEILING function. The number of extra stages upper bound then becomes:

$$\begin{aligned} Number_Extra_Stages_p &= (p-2) \text{CEILING} \left[\log_2 (N_{cols}) \right]. \end{aligned} \quad (20)$$

Table 8 shows the common series and extra stages for power variable p from 2 (at Sequence 1) up to general value p . The total sum of common stages at p is simply $(p-1)N_{cols}$. The values in the Number_Extra_Stages form an arithmetic

TABLE 8. Total Stages by Power Variable p , with $N_p = N_{cols}^p$.

p	Seq. q	Common Seq. 1 Stages	Number Extra Stages
2	1	N_{cols}	0
3	2	N_{cols}	$1 \cdot \text{CEILING}(\log_2(N_{cols}))$
4	3	N_{cols}	$2 \cdot \text{CEILING}(\log_2(N_{cols}))$
5	4	N_{cols}	$3 \cdot \text{CEILING}(\log_2(N_{cols}))$
...			
p	$p-1$	N_{cols}	$(p-2) \text{CEILING}(\log_2(N_{cols}))$

series, with the $\text{CEILING}(\log_2 N_{cols})$ being the constant difference between values in the series. The total number of stages in this column is then:

$$\begin{aligned} Total_Number_Extra_Stages_p &= \frac{1}{2}(p-2)(1+p-2) \text{CEILING}(\log_2(N_{cols})) \\ &= \frac{1}{2}(p-2)(p-1) \text{CEILING}(\log_2(N_{cols})). \end{aligned} \quad (21)$$

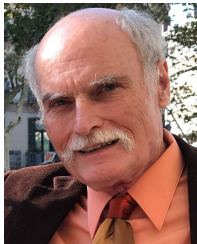
Finally, when the single Sequence 0 stage, the series of common stages, and the series of extra stages are added together, the resultant $Total_Stages$ equation matches Equation (18):

$$Total_Stages = 1 + (p-1)N_{cols} + \frac{1}{2}(p-2)(p-1) \text{CEILING}(\log_2 N_{cols}).$$

REFERENCES

- [1] D. E. Knuth, *The Art of Computer Programming: Sorting and Searching*, vol. 3. London, U.K.: Pearson Education, 1997.
- [2] R. B. Kent and M. S. Pattichis, "Design, implementation, and analysis of high-speed single-stage N-sorters and N-filters," *IEEE Access*, vol. 9, pp. 2576–2591, 2021.
- [3] R. Kent and M. Pattichis, "Use of carry chain logic and design system extensions to construct significantly faster and larger single-stage N-sorters and N-filters," *TechRxiv*, Mar. 28, 2022, doi: 10.36227/techrxiv.19400867.v2.
- [4] K. E. Batcher, "Sorting networks and their applications," in *Proc. Spring Joint Comput. Conf.*, vol. 2, New York, NY, USA, 1968, pp. 307–314, doi: 10.1145/1468075.1468121.
- [5] I. Skliarova and V. Sklyarov, "Reconfigurable devices and design tools," in *FPGA-BASED Hardware Accelerators*. Cham, Switzerland: Springer, 2019, pp. 1–38.
- [6] A. Adams, "Fast median filters using separable sorting networks," *ACM Trans. Graph.*, vol. 40, no. 4, pp. 1–11, Aug. 2021.
- [7] A. Farmahini-Farahani, H. J. Duwe, M. J. Schulte, and K. Compton, "Modular design of high-throughput, low-latency sorting units," *IEEE Trans. Comput.*, vol. 62, no. 7, pp. 1389–1402, Jul. 2013.
- [8] M. Saitoh, E. A. Elsayed, T. V. Chu, S. Mashimo, and K. Kise, "A high-performance and cost-effective hardware merge sorter without feedback datapath," in *Proc. IEEE 26th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, Apr. 2018, pp. 197–204.
- [9] N. Satish, C. Kim, J. Chhugani, D. Nguyen, V. W. Lee, D. Kim, and P. Dubey, "Fast sort on CPUs and GPUs: A case for bandwidth oblivious SIMD sort," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Jun. 2010, pp. 351–362.
- [10] N. Satish, M. Harris, and M. Garland, "Designing efficient sorting algorithms for manycore GPUs," in *Proc. IEEE Int. Symp. Parallel Distrib. Process.*, May 2009, pp. 1–10.
- [11] D.-L. Lee and K. E. Batcher, "A multiway merge sorting network," *IEEE Trans. Parallel Distrib. Syst.*, vol. 6, no. 2, pp. 211–215, Feb. 1995, doi: 10.1109/71.342136.

- [12] K. J. Liszka and K. E. Batchner, "A modulo merge sorting network," in *Proc. 4th Symp. Frontiers Massively Parallel Comput.*, 1992, pp. 164–169, doi: [10.1109/FMPC.1992.234892](https://doi.org/10.1109/FMPC.1992.234892).
- [13] D. C. Van Voorhis, "An economical construction for sorting networks," in *Proc. Nat. Comput. Conf. Exp. (AFIPS)*, May 1974, pp. 921–927, doi: [10.1145/1500175.1500347](https://doi.org/10.1145/1500175.1500347).
- [14] Q. Gao and Z. Liu, "Sloping-and-shaking—Multiway merging and sorting," *Sci. China E, Technol. Sci.*, vol. 40, no. 3, pp. 225–234, Mar. 1997.
- [15] P. Kolte, "A fast median filter using AltiVec," in *Proc. IEEE Int. Conf. Comput. Design, VLSI Comput. Processors*, Oct. 1999, pp. 384–391, doi: [10.1109/ICCD.1999.808571](https://doi.org/10.1109/ICCD.1999.808571).
- [16] P. Kolte and R. Smith, "SIMD computation of rank based filters for M×N grids," U.S. Patent 6 058 405, May 2, 2000.
- [17] R. Osumi, "Ultrasonic diagnostic apparatus and image filtering method of the same," U.S. Patent 8 098 913, Jan. 17, 2012.
- [18] D. Gale and R. M. Karp, "A phenomenon in the theory of sorting," in *Proc. 11th Annu. Symp. Switching Automata Theory (SWAT)*, 1970, pp. 51–59, doi: [10.1109/SWAT.1970.1](https://doi.org/10.1109/SWAT.1970.1).
- [19] *Amazon EC2 Instance Types*. Accessed: Feb. 14, 2022. [Online]. Available: <https://aws.amazon.com/ec2/instance-types/>
- [20] *Developing on AWS F1 With SDAccel and RTL Kernels—Part 2 of 4*. Accessed: Feb. 16, 2022. [Online]. Available: <https://youtu.be/UQFAZyUgKIs>



ROBERT B. KENT (Life Member, IEEE) received the B.S. degree in physics from the University of Notre Dame, Notre Dame, IN, USA, in 1970, and the M.S. degree in electrical engineering from The University of Utah, Salt Lake City, UT, USA, in 1983. He is currently pursuing the Ph.D. degree with the Electrical and Computer Engineering Department, The University of New Mexico.

He worked for various semiconductor companies, such as National Semiconductor, from 1983 to 1990, Intel Corporation, from 1990 to 1998, Philips Semiconductor, from 1998 to 1999, and Xilinx Inc., from 1999 to 2011. He then worked as an Independent Contractor, also in the semiconductor field, from 2012 to 2017. His main research interests include the design of single-stage N-sorters and N-filters in hardware, particularly in FPGAs, and the use of these sorters and filters in sorting networks or other hardware sorting systems.



MARIOS S. PATTICHIS (Senior Member, IEEE) received the B.Sc. degree (Hons.) in computer sciences, the Bachelor of Arts degree (Hons.) in mathematics, and a minor in electrical engineering, the M.S. degree in electrical engineering, and the Ph.D. degree in computer engineering from The University of Texas at Austin, in 1991, 1993, and 1998, respectively.

For his development of the digital logic design labs at The University of New Mexico (UNM), he was recognized by Xilinx Corporation, in 2003, and by the UNM School of Engineering's Harrison Faculty Excellence Award in 2006. He was the Lead PI and a Board Member of the Configurable Space and Microsystems Innovations and Applications Center (COSMIAC), UNM. At UNM, he also works as the Director of the Image and Video Processing and Communications Laboratory (ivPCL). He is currently a Professor with the Department of Electrical and Computer Engineering, UNM. He holds the 2019–2022 ECE Gardner Zemke Professorship for teaching. His current research interests include image and video processing, video communications, dynamically reconfigurable computer architectures, and biomedical image analysis.

Dr. Pattichis was a fellow of the Center for Collaborative Research and Community Engagement, UNM College of Education, from 2019 to 2020. In 2022, he was elected as a fellow of the European Alliance of Medical and Biological Engineering and Science (EAMBES). He was a recipient of the 2016 Lawton-Ellis and the 2004 Distinguished Teaching Awards from the Department of Electrical and Computer Engineering, UNM. He was the General Chair of the 2008 IEEE Southwest Symposium on Image Analysis and Interpretation (SSIAI) and served as a General Co-Chair for the SSIAI in 2020. He is also serving as a Guest Associate Editor for the Special Issue on "Large Scale Video Analytics for Clinical Decision Support," to be published by the IEEE JOURNAL OF BIOMEDICAL AND HEALTH INFORMATICS and for the Special Issue on "Teaching and Learning Mathematics and Computing in Multilingual Contexts," to be published by *Teachers College Record*. He has served as a Senior Associate Editor for the IEEE TRANSACTIONS ON IMAGE PROCESSING and the IEEE SIGNAL PROCESSING LETTERS, an Associate Editor for the IEEE TRANSACTIONS ON IMAGE PROCESSING and the IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, and a Guest Associate Editor for two additional Special Issues published in the IEEE TRANSACTIONS ON INFORMATION TECHNOLOGY IN BIOMEDICINE, and another Special Issue published in *Biomedical Signal Processing and Control*.

...